POSTGRESQL

Isolation Levels

Sıfırdan, terminləri bir-bir izah edən tam və aydın bələdçi

YENİLƏNMİŞ GENİŞ VERSİYA

Database əsasları → Transaction → Snapshot → MVCC

Dirty read → Phantom → Lost update → Write skew

Read Committed → Repeatable Read → Serializable

Locks → Retry → EF Core → ECafe nümunəsi → Tam lüğət

Hazırlanma tarixi: 24 sentyabr 2026

1. Bu sənədi necə oxumalısan?

Bütün anlayışlar əvvəl sadə dillə, sonra texniki dillə izah edilir.

Isolation level-i anlamaq üçün birbaşa dörd səviyyəni əzbərləmək düzgün yol deyil. Əvvəl transaction-un nə olduğunu, sonra paralel transaction-ların hansı problemləri yaratdığını anlamaq lazımdır.

Mərhələ	Öyrənəcəyin şey	Niyə lazımdır?
1. Baza	Database, row, query, transaction	Sonrakı terminlərin dilidir
2. Görünüş	Snapshot və MVCC	Transaction-un nə gördüyünü izah edir
3. Problemlər	Dirty read, phantom, lost update, write skew	Isolation level-in nəyi həll etdiyini göstərir
4. Səviyyələr	Read Committed, Repeatable Read, Serializable	Düzgün səviyyəni seçməyə imkan verir
5. Tətbiq	Lock, retry, EF Core, ECafe	Nəzəriyyəni real koda çevirir

> **Oxuma qaydası**
> Qalın yazılmış ingilis termini görəndə həmin səhifədə onun sadə mənası verilir. Sənədin sonunda bütün terminlərin Azərbaycan dilində geniş lüğəti də var.

2. Ən əsas database terminləri

Isolation mövzusuna başlamazdan əvvəl istifadə edilən dil.

Database - verilənlər bazası

Sadə izah: Tətbiqin məlumatlarını nizamlı saxlayan sistemdir.
Texniki izah: PostgreSQL database server-dir; cədvəlləri, constraint-ləri və transaction-ları idarə edir.
Nümunə: ECafe database-də restoran, masa, rezervasiya və istifadəçi məlumatları saxlanılır.

Table - cədvəl

Sadə izah: Eyni tip məlumatların saxlandığı siyahıdır.
Texniki izah: Sütunlardan və sətirlərdən ibarət database obyektidir.
Nümunə: Reservations cədvəli bütün rezervasiyaları saxlayır.

Row - sətir

Sadə izah: Cədvəldə bir məlumat obyektidir.
Texniki izah: Bir entity-nin database-dəki konkret qeydidir.
Nümunə: Reservations cədvəlində ID-si 22 olan rezervasiya bir row-dur.

Column - sütun

Sadə izah: Row-un bir xüsusiyyətidir.
Texniki izah: Cədvəlin məlumat tipi və qaydası olan sahəsidir.
Nümunə: StatusId, ReservedAt və TableId sütunlardır.

Query - sorğu

Sadə izah: Database-ə verilən oxu və ya dəyişiklik əmridir.
Texniki izah: SELECT, INSERT, UPDATE və DELETE SQL statement-ləridir.
Nümunə: SELECT * FROM Reservations məlumat oxuyan query-dir.

3. Request, connection və session

Bir-birinə oxşayan, amma fərqli üç anlayış.

Request - sorğu

Sadə izah: Müştərinin tətbiqə göndərdiyi bir istəkdir.
Texniki izah: HTTP request controller və service kodunu işə salır.
Nümunə: POST /reservations yeni rezervasiya yaratmaq istəyidir.

Connection - bağlantı

Sadə izah: Tətbiqlə PostgreSQL arasındakı rabitə xəttidir.
Texniki izah: NpgsqlConnection database server-ə fiziki və ya pooled bağlantını təmsil edir.
Nümunə: EF Core query göndərmək üçün connection istifadə edir.

Session - database sessiyası

Sadə izah: PostgreSQL-in bir aktiv bağlantı üçün saxladığı vəziyyətdir.
Texniki izah: Connection açıldıqda server tərəfində backend process/session yaranır.
Nümunə: Session-level advisory lock həmin session-a bağlı qalır.

Bunlar eyni deyil

Bir HTTP request bir transaction açа bilər, amma məcbur deyil. Connection pool səbəbindən fiziki database session sonrakı request-lərdə yenidən istifadə oluna bilər.

Isolation level HTTP request-ə yox, database transaction-a tətbiq olunur.

4. Transaction nədir?

Bir neçə database əməliyyatını vahid iş kimi idarə edən sərhəd.

> **Transaction**
> Sadə izah: Birlikdə uğurlu olmalı database əməliyyatlarının paketidir.
> Texniki izah: BEGIN ilə başlayır, COMMIT və ya ROLLBACK ilə bitir.
> Nümunə: Rezervasiya, status tarixçəsi və ödəniş qeydi birlikdə yazılırsa eyni transaction-da ola bilər.

```
BEGIN;

INSERT INTO reservations (...);
INSERT INTO reservation_status_history (...);

COMMIT;
```

Əmr	Sadə mənası	Nəticə
BEGIN	İş paketini başlat	Transaction aktiv olur
COMMIT	Dəyişiklikləri rəsmiləşdir	Digər transaction-lar nəticəni görə bilər
ROLLBACK	Bu paketdəki dəyişiklikləri ləğv et	Database transaction-dan əvvəlki vəziyyətə qayıdır

> **Vacib**
> Transaction içindəki xarici e-mail, MinIO upload və bank əməliyyatı database rollback ilə avtomatik geri qayıtmır. Buna görə outbox və idempotency ayrıca lazımdır.

5. Commit və rollback

Transaction-un nəticəsini müəyyən edən iki sonluq.

Commit

Sadə izah: Transaction-da edilən dəyişiklikləri təsdiqləməkdir.
Texniki izah: Commit-dən sonra dəyişikliklər digər transaction-ların snapshot qaydasına uyğun görünür.
Nümunə: Rezervasiya row-u commit-dən sonra digər request-lər üçün mövcuddur.

Rollback

Sadə izah: Transaction-da edilən database dəyişikliklərini geri atmaqdır.
Texniki izah: Exception, cancellation və ya explicit qərar transaction-u abort edə bilər.
Nümunə: Rezervasiya yazıldıqdan sonra xəta çıxarsa rollback həmin row-u saxlamır.

Sual	Cavab	Səbəb
Rollback edilmiş INSERT başqa request-ə görünür?	Xeyr	Heç vaxt commit olunmayıb
Commit olunmuş dəyişiklik dərhal hamıya görünür?	Level-dən asılıdır	Repeatable Read köhnə snapshot-u saxlaya bilər
Sequence dəyəri rollback olur?	Xeyr	PostgreSQL sequence dəyişiklikləri transaction rollback-ına tabe deyil

Deməli, “commit olunub” və “mənim transaction-ım bunu görür” eyni cümlə deyil. Görünürlük isolation level-dən asılıdır.

6. Concurrency nədir?

İki və ya daha çox işin zaman baxımından üst-üstə düşməsi.

Concurrency - paralel işləmə

Sadə izah: Bir neçə request-in eyni vaxt aralığında işləməsidir.
Texniki izah: Əməliyyatlar CPU-da tam eyni nanosaniyədə işləməsə belə addımları bir-birinə qarışa bilər.
Nümunə: İki müştəri eyni masanı eyni anda rezervasiya edir.

Race condition - yarış vəziyyəti

Sadə izah: Nəticənin request-lərin hansı sırada işləməsindən asılı olmasıdır.
Texniki izah: Check ilə write arasında başqa transaction vəziyyəti dəyişir.
Nümunə: Hər iki request "masa boşdur" görüb iki rezervasiya yaradır.

Vaxt	Request A	Request B
T1	Masa boşdur? Bəli	
T2		Masa boşdur? Bəli
T3	Rezervasiya yazır	
T4		Rezervasiya yazır

Nəticə

Kod tək request-lə test ediləndə düzgün görünür. Problem yalnız iki request-in addımları qarışanda yaranır.

7. Invariant nədir?

Sistemdə həmişə doğru qalmalı biznes qaydası.

Invariant - dəyişməz biznes qaydası

Sadə izah: Əməliyyatlar bitsə də pozulmamalı şərtdir.
Texniki izah: Transaction consistency dizaynının qorumağa çalışdığı domain qaydasıdır.
Nümunə: Eyni masa və vaxt üçün maksimum bir aktiv rezervasiya olmalıdır.

Invariant	Pozularsa nə olar?	Mümkün müdafiə
Masa/time slot üzrə bir rezervasiya	Double booking	Constraint və ya advisory lock
Balans mənfi olmamalıdır	Pul qaydası pozulur	Atomic UPDATE / row lock
Ən azı bir menecer aktiv qalmalıdır	İdarə edən şəxs qalmır	Serializable və ya parent row lock
Customer/day maksimum bir rezervasiya	Spam və biznes limiti pozulur	Customer/day lock + re-check

Isolation level seçiminin başlanğıcı

Əvvəl level seçilmir. Əvvəl invariant yazılır, sonra onu poza biləcək parallel read/write ssenarisi çəkilir.

8. Snapshot nədir?

Database-in müəyyən andakı görünən vəziyyətinin şəkli.

Snapshot

Sadə izah: Transaction-un hansı məlumat versiyalarını görə bildiyini müəyyən edən "şəkildir".
Texniki izah: Commit olunmuş transaction-ların və aktiv transaction-ların vəziyyətinə əsasən row visibility hesablanır.
Nümunə: Saat 10:00-da snapshot alan transaction sonrakı commit-ləri level-dən asılı olaraq görməyə bilər.

Foto analoqu

Restoran zalının şəklini çəkdin. Sonra bir masa tutuldu. Köhnə fotoda masa yenə boş görünür, real zalda isə doludur.
Snapshot real vaxt deyil; qaydaya görə seçilmiş görünüşdür.

Level	Snapshot nə vaxt alınır?	Nə qədər yaşayır?
Read Committed	Hər SQL statement başlayanda	Yalnız həmin statement müddətində
Repeatable Read	Transaction-un ilk real query/write statement-ində	Transaction bitənədək
Serializable	Repeatable Read kimi	Transaction bitənədək + dependency yoxlaması

9. MVCC nədir?

PostgreSQL-in oxu və yazıları sətir versiyaları ilə idarə etməsi.

MVCC - Multi-Version Concurrency Control

Sadə izah: Eyni məlumatın köhnə və yeni versiyalarını müvəqqəti saxlayaraq paralel oxuma-yazmaya imkan verən mexanizmdir.
Texniki izah: UPDATE yeni row version yaradır; snapshot uyğun versiyanı seçir.
Nümunə: A köhnə qiyməti oxuyarkən B yeni qiyməti yazıb commit edə bilər.

Tuple version - sətir versiyası

Sadə izah: Bir row-un zaman üzrə konkret variantıdır.
Texniki izah: PostgreSQL UPDATE zamanı köhnə tuple-ı görünməzləşdirib yenisini yaradır.
Nümunə: Status Pending-dən Confirmed-ə dəyişəndə yeni row version yaranır.

Visibility - görünürlük

Sadə izah: Snapshot-un hansı row versiyasını göstərəcəyidir.
Texniki izah: xmin/xmax və transaction vəziyyəti ilə hesablanır.
Nümunə: Köhnə transaction köhnə versiyanı, yeni transaction yeni versiyanı görə bilər.

MVCC bütün problemi həll etmir

MVCC oxucunun writer-i lazımsız bloklamasını azaldır. Amma biznes invariantı, lost update və write skew üçün əlavə qayda lazım ola bilər.

10. Isolation level nədir?

Transaction-un digər transaction-ların dəyişikliklərini necə görəcəyini müəyyən edən rejim.

Isolation level

Sadə izah: Paralel transaction-lar arasında görünürlük və konflikt qaydalarının səviyyəsidir.
Texniki izah: SQL standartı dörd level adlandırır; PostgreSQL üç fərqli real davranış verir.
Nümunə: Read Committed hər query-də təzə snapshot, Repeatable Read transaction boyu sabit snapshot verir.

Səhv fikir	Düz fikir
Yüksək level həmişə daha yaxşıdır	Yüksək level daha sərt zəmanət, amma retry və overhead gətirə bilər
Isolation hər şeyi kilidləyir	PostgreSQL əsasən MVCC snapshot istifadə edir
Serializable bütün request-ləri sıra ilə işlədir	Fiziki paralellik qalır; uyğunsuz nəticə yaradan transaction abort edilir
Read Committed təhlükəlidir	Düzgün lock, re-check və constraint ilə çox flow üçün doğru default-dur

Əsas sual

Transaction A iki dəfə query göndərsə, arada B commit etdikdə A ikinci dəfə yeni nəticəni görməlidir, yoxsa əvvəlki görünüş sabit qalmalıdır?

11. Anomaly nədir?

Paralel transaction-ların yaratdığı arzuolunmaz və ya serial izahı olmayan nəticə.

Anomaly - anomaliya

Sadə izah: Concurrency səbəbindən yaranan qəribə və ya biznescə səhv nəticədir.
Texniki izah: Isolation nəzəriyyəsində müəyyən read/write pattern-lərinə verilən addır.
Nümunə: Eyni balansdan iki dəfə pul çıxılıb, amma biri itib.

Anomaliya	Bir cümləlik məna
Dirty read	Başqasının commit etmədiyi məlumatı oxumaq
Non-repeatable read	Eyni row-u təkrar oxuyanda fərqli dəyər görmək
Phantom read	Eyni filtr sorğusunda row siyahısının dəyişməsi
Lost update	Bir yazının digər yazının nəticəsini silməsi
Write skew	Fərqli row yazılarının ümumi qaydanı birlikdə pozması
Serialization anomaly	Yekun nəticənin heç bir ardıcıl icra sırası ilə izah edilə bilməməsi

Növbəti səhifələr

Hər anomaliya ayrıca addım-addım izah olunur. Terminləri cədvəldən əzbərləmək yox, iki transaction-un nə gördüyünü izləmək lazımdır.

12. Dirty read

Başqa transaction-un hələ commit etmədiyi məlumatı oxumaq.

Dirty read

Sadə izah: Hələ təsdiqlənməmiş və rollback ola biləcək məlumatı görməkdir.
Texniki izah: Transaction B, A-nın uncommitted row versiyasını oxuyur.
Nümunə: A balansı 0 edir, B 0 görür, sonra A rollback edir.

```
Transaction A                Transaction B
UPDATE balance = 0;          SELECT balance; -- 0 görsə dirty read
-- COMMIT yoxdur
ROLLBACK;
```

Sual	Cavab	Səbəb
PostgreSQL-də dirty read mümkündür?	Xeyr	Read Uncommitted də Read Committed kimi işləyir
A öz commit olunmamış yazısını görür?	Bəli	Transaction öz dəyişikliklərini görür
B A-nın commit olunmamış yazısını görür?	Xeyr	Başqa transaction üçün görünməzdir

Niyə təhlükəlidir?

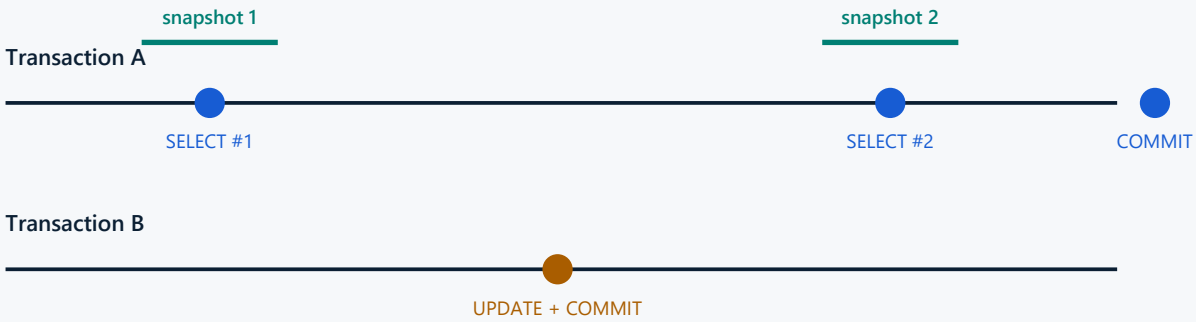
B dirty məlumat əsasında notification və ya bank əməliyyatı etsə, A rollback etdikdən sonra qərarın əsası database-də heç vaxt mövcud olmamış olacaq.

13. Non-repeatable read

Eyni row-u transaction daxilində ikinci dəfə oxuyanda fərqli dəyər görmək.

Non-repeatable read

Sadə izah: Birinci oxuduğun dəyəri ikinci oxuda təkrar görə bilməməkdir.
Texniki izah: Arada başqa transaction həmin row-u update edib commit edir.
Nümunə: Status əvvəl Pending, ikinci oxuda Confirmed görünür.



Addımlar:

- A row-u oxuyur: status = Pending.
- B status-u Confirmed edir və commit edir.
- A eyni row-u yenidən oxuyur: Read Committed-də Confirmed görə bilər.

Bu həmişə xəta deyil

Ən yeni statusu görmək istəyən qısa API əməliyyatında bu davranış faydalıdır. Sabit hesablama istəyən transaction üçün isə Repeatable Read lazım ola bilər.

14. Phantom read

Eyni filter query-ni təkrarlayanda nəticə siyahısına row gəlməsi və ya çıxması.

Predicate - filter şərti

Sadə izah: Hansı row-ların seçiləcəyini müəyyən edən qaydadır.
Texniki izah: SQL WHERE hissəsidir.
Nümunə: WHERE RestaurantId = 1 AND Status = 'Active'.

Phantom read

Sadə izah: Eyni filter query-nin ikinci dəfə fərqli row siyahısı qaytarmasıdır.
Texniki izah: Başqa transaction predicate-ə uyğun row insert/delete edib commit edir.
Nümunə: Birinci query 4 rezervasiya, ikinci query 5 rezervasiya qaytarır.

```
A: SELECT COUNT(*) FROM reservations WHERE day = '2026-09-24'; -- 4
B: INSERT uyğun rezervasiya; COMMIT;
A: SELECT COUNT(*) FROM reservations WHERE day = '2026-09-24'; -- 5
```

Sual	Cavab	Səbəb
Read Committed-də phantom mümkündür?	Bəli	İkinci query yeni snapshot alır
PostgreSQL Repeatable Read-də phantom mümkündür?	Xeyr	Transaction eyni snapshot-u saxlayır
Phantom yoxdursa bütün biznes qaydaları qorunur?	Xeyr	Write skew yenə mümkündür

15. Lost update

İki yazıdan birinin nəticəsinin səssizcə itməsi.

Lost update
Sadə izah: İki request eyni köhnə dəyərdən hesablayır və son yazan əvvəlkinin nəticəsini silir.
Texniki izah: Read-modify-write pattern-də concurrency control olmadıqda yaranır.
Nümunə: Balans 100-dür; iki request 30 çıxır; son balans səhvən 70 qalır.

Addım	Request A	Request B
1	100 oxuyur	100 oxuyur
2	100 - 30 = 70	100 - 30 = 70
3	70 yazır	70 yazır
Nəticə	İki dəfə 30 çıxılıb	Database-də yalnız 30 azalma görünür

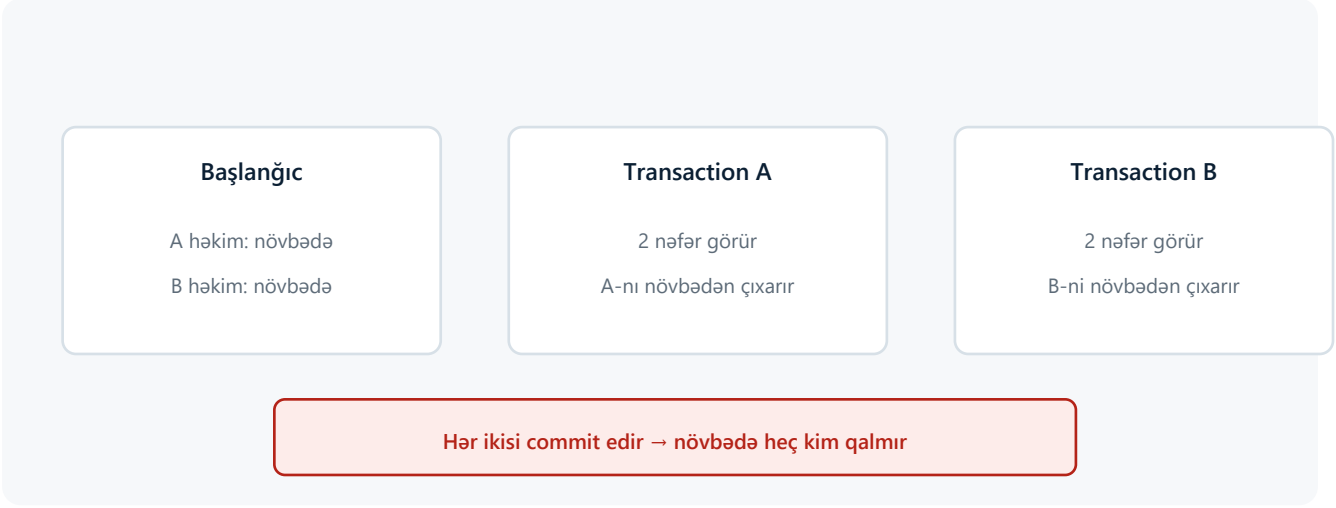
Həll variantları:

- Atomic UPDATE: SET balance = balance - 30.
- SELECT FOR UPDATE ilə row-u əvvəl kilidləmək.
- Optimistic concurrency token ilə versiya dəyişməsini aşkarlamaq.
- Uyğun hallarda Serializable və full retry.

16. Write skew

Fərqli row-lara yazan transaction-ların ümumi invariantı pozması.

Write skew
Sadə izah: Hər transaction ayrıca düzgün qərar verir, amma birlikdə nəticə biznes qaydasını pozur.
Texniki izah: Transaction-lar eyni predicate-i oxuyur, fərqli row-lara yazır və direct row conflict yaranmır.
Nümunə: İki aktiv menecer ayrı-ayrılıqda özünü deaktiv edir və aktiv menecer qalmır.



Niyə Repeatable Read bunu dayandırmır?

- A sabit snapshot-da 2 aktiv menecer görür.
- B də öz sabit snapshot-ında 2 aktiv menecer görür.
- A Menecer-1 row-unu, B Menecer-2 row-unu dəyişir.
- Eyni row-a yazmadıqları üçün hər ikisi commit edə bilər.

Həll
Serializable dependency-ni aşkarlayıb bir transaction-u 40001 ilə dayandıra bilər. Alternativ olaraq ümumi parent row və ya advisory lock ilə invariant serial edilə bilər.

17. Serialization anomaly

Yekun nəticənin heç bir ardıcıl icra sırası ilə izah edilə bilməməsi.

Serial execution - ardıcıl icra

Sadə izah: Transaction-ların biri tam bitdikdən sonra digərinin başlamasıdır.
Texniki izah: Concurrency olmayan nəzəri sıra modelidir.
Nümunə: Əvvəl A tam işləyir, sonra B.

Serialization anomaly

Sadə izah: Paralel nəticənin A→B və ya B→A ardıcıllıqlarının heç birinə uyğun olmamasıdır.
Texniki izah: Committed transaction set üçün serial order tapmaq mümkün deyil.
Nümunə: Write skew nəticəsində iki menecer də deaktivdir; halbuki ardıcıl işləsəydilər ikinci transaction bunu etməzdi.

İcra sırası	İkinci transaction nə görərdi?	Yekun
Əvvəl A, sonra B	B yalnız bir aktiv menecer görərdi	B deaktiv etməməli idi
Əvvəl B, sonra A	A yalnız bir aktiv menecer görərdi	A deaktiv etməməli idi
Paralel Repeatable Read	Hər ikisi iki nəfər görür	Hər ikisi deaktiv edə bilir

Əsas fərq

Phantom “mən transaction daxilində nə gördüm?” sualıdır. Serialization anomaly isə “bütün transaction-lar bitəndə yaranan yekun nəticə ardıcıl icra ilə mümkündürmü?” sualıdır.

18. Phantom yoxdur, anomaly mümkündür

Ən çox qarışdırılan cümlənin masa nümunəsi ilə tam izahı.

Restoranda iki boş masa var və biznes qaydası deyir: walk-in müştərilər üçün ən azı bir masa boş qalmalıdır.

Addım	Transaction A	Transaction B
1	2 boş masa görür	2 boş masa görür
2	Masa-1-i rezervasiya edir	Masa-2-ni rezervasiya edir
3	Öz snapshot-ında əvvəlki siyahı sabitdir	Öz snapshot-ında əvvəlki siyahı sabitdir
4	Commit	Commit
Yekun	Boş masa 0	Biznes qaydası pozuldu

Niyə phantom yoxdur?

A öz transaction-ında query-ni təkrar etsə, snapshot dəyişmədiyi üçün yenə iki boş masa görür. Nəticə siyahısına yeni row daxil olmur və çıxmır.

Niyə serialization anomaly var?

Hər ikisi commit etdikdən sonra boş masa qalmır. Ardıcıl işləsəydilər ikinci transaction yalnız bir boş masa görəcək və rezervasiya yaratmamalı idi.

19. Read Uncommitted

PostgreSQL-də ayrıca zəif davranış verməyən standart adı.

Read Uncommitted

Sadə izah: SQL standartında başqa transaction-un commit etmədiyi məlumatı görə bilən ən zəif level-dir.
Texniki izah: PostgreSQL bu adı Read Committed davranışına xəritələyir.
Nümunə: PostgreSQL-də READ UNCOMMITTED yazsan da dirty read baş vermir.

Sual	Cavab	Səbəb
Dirty read mümkündür?	PostgreSQL-də xeyr	Read Committed kimi işləyir
Non-repeatable read mümkündür?	Bəli	Hər statement yeni snapshot ala bilər
Phantom mümkündür?	Bəli	İkinci query yeni commit-ləri görə bilər

Nəticə

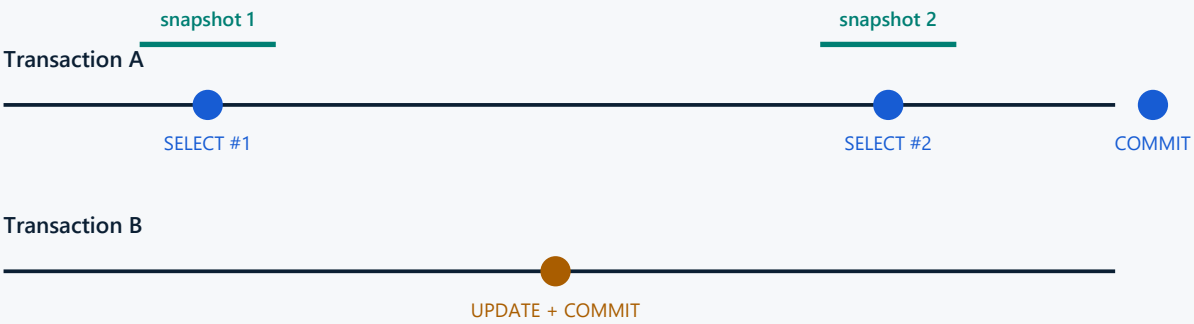
PostgreSQL layihəsində performans üçün Read Uncommitted seçməyin xüsusi faydası yoxdur. Adı görmək çox vaxt başqa database-dən daşınmış yanlış fərziyyədir.

20. Read Committed

PostgreSQL default-u: hər statement üçün yeni snapshot.

Read Committed

Sadə izah: Query başlayanda artıq commit olunmuş məlumatları görən level-dir.
Texniki izah: Hər SQL statement öz snapshot-unu alır; transaction boyu snapshot sabit deyil.
Nümunə: Birinci SELECT Pending, ikinci SELECT aradakı commit-dən sonra Confirmed görə bilər.



Qoruyur	Qorumur
Dirty read baş vermir	Non-repeatable read mümkündür
Bir statement daxilində görüntü sabitdir	Phantom mümkündür
Qısa CRUD üçün yaxşı concurrency verir	Check-then-act özü avtomatik təhlükəsiz deyil

Sadə fikir

Read Committed deyir: "Bu query başlayanda təsdiqlənmiş nə varsa onu göstər." Növbəti query başlayanda dünya dəyişmiş ola bilər.

21. Read Committed nə vaxt yaxşıdır?

Düzgün əlavə mexanizmlərlə əksər qısa API transaction-ları üçün.

Ssenari	Read Committed ilə necə təhlükəsiz edilir?
Sadə INSERT	UNIQUE və digər constraint-lər
Status dəyişmə	Conditional UPDATE və affected row yoxlaması
Balans azaltma	Atomic UPDATE və ya FOR UPDATE
Masa rezervasiyası	Advisory lock → re-check → INSERT
Entity redaktəsi	Optimistic concurrency token

Check-then-act

Sadə izah: Əvvəl şərti yoxlayıb sonra həmin nəticəyə əsasən yazmaqdır.
Texniki izah: Yoxlama ilə yazı arasında concurrency pəncərəsi yaranır.
Nümunə: Masa boşdurmu? Bəli → rezervasiya yarat.

Qayda

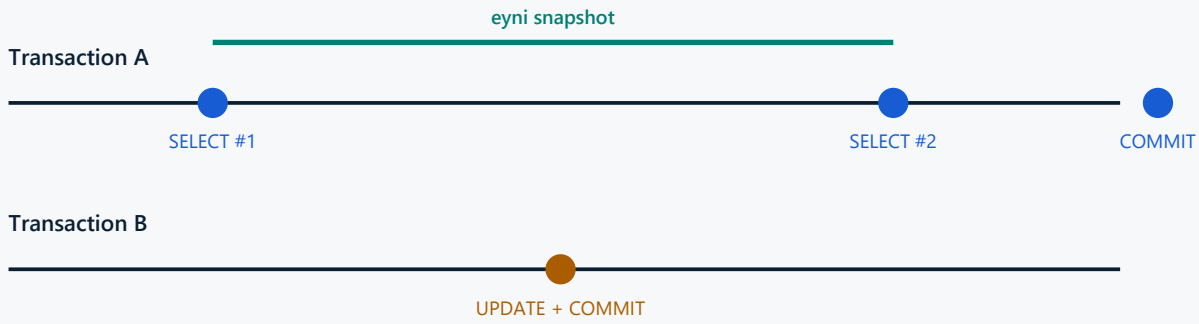
Check-then-act flow-da lock əvvəl alınmalı, vəziyyət lock-dan sonra yenidən yoxlanmalı və yazı eyni transaction-da commit edilməlidir.

22. Repeatable Read

Transaction boyu eyni snapshot-u saxlayan level.

Repeatable Read

Sadə izah: Transaction daxilində əvvəl gördüyün database görünüşünü sabit saxlayır.
Texniki izah: Snapshot ilk real statement zamanı alınır və transaction sonuna qədər istifadə olunur.
Nümunə: İkinci SELECT aradakı commit-i görmür.



Qoruyur	Qorumur / xərc
Dirty read yoxdur	Ən yeni məlumatı transaction boyu göstərmir
Non-repeatable read yoxdur	Write skew mümkündür
PostgreSQL-də phantom yoxdur	Concurrent update zamanı 40001 yarana bilər

Sadə fikir

Repeatable Read deyir: “Transaction-un əvvəlində gördüyün şəkli sona qədər saxla.” Şəkil sabitdir, amma paralel transaction-ların yekun birgə nəticəsi yenə səhv ola bilər.

23. Repeatable Read nə vaxt seçilir?

Sabit görünüş lazımdır, amma tam serial invariant zəmanəti lazım deyil.

Uyğun nümunələr:

- Bir neçə query-dən ibarət maliyyə və ya audit hesabatı.
- Eyni transaction daxilində header və detail məlumatlarının eyni zaman nöqtəsinə aid olması.
- Uzun olmayan, sabit oxu tələb edən analitik əməliyyat.

Uyğun olmayan və ya əlavə müdafiə istəyən nümunələr:

- Cross-row invariant və write skew riski.
- Ən son statusu hər query-də görmək tələb olunan əməliyyat.
- Retry edə bilməyən xarici side effect-li transaction.

Serialization failure - 40001

Sadə izah: Database-in transaction-u təhlükəsiz davam etdirə bilmədiyi üçün dayandırmasıdır.
Texniki izah: PostgreSQL SQLSTATE 40001 qaytarır; tam transaction yenidən başlamalıdır.
Nümunə: Repeatable Read-də concurrent update eyni row-a toxunarsa yarana bilər.

24. Serializable

Uğurlu commit-lərin nəticəsini ardıcıl icra kimi zəmanət edən ən sərt level.

Serializable

Sadə izah: Paralel transaction-ların yekununu sanki bir-bir işləyiblər kimi qoruyan level-dir.
Texniki izah: PostgreSQL snapshot + SSI dependency analysis istifadə edir.
Nümunə: Write skew yaradacaq iki transaction-dan biri abort edilir.

Nə baş verir?	İzah
Transaction-lar paralel işləyir	Serializable onları fiziki olaraq tam sıra ilə işlətmir
Read/write əlaqələri izlənir	Təhlükəli dependency pattern axtarılır
Serial nəticə mümkün deyilsə	Transaction-lardan biri 40001 ilə abort edilir
Tətbiq retry edir	Transaction bütün qərar məntiqi ilə yenidən başlayır

Sadə fikir

Serializable deyir: “Hamınız paralel işləyə bilərsiniz, amma nəticəniz ardıcıl işləsəydiniz mümkün olmayacaqsa birinizi commit etməyəcəyəm.”

25. SSI və predicate lock

Serializable-in daxilində işləyən PostgreSQL mexanizmi.

SSI - Serializable Snapshot Isolation

Sadə izah: Snapshot üzərində işləyib təhlükəli read/write əlaqələrini aşkarlayan alqoritmdir.
Texniki izah: Serializable nəticəni qorumaq üçün dependency graph haqqında məlumat toplayır.
Nümunə: A-nın oxuduğu nəticəyə B-nin yazısının təsir edib-etməyəcəyini izləyir.

Predicate lock / SIReadLock

Sadə izah: Transaction-un oxuduğu şərti database-in yadda saxlamasıdır.
Texniki izah: PostgreSQL pg_locks-də SIReadLock kimi görünə bilər; adi lock kimi writer-i bloklamır.
Nümunə: A "bütün aktiv menecerlər" query-sini oxuyubsa həmin predicate dependency analizi üçün qeyd edilir.

Dependency - asılılıq

Sadə izah: Bir transaction-un nəticəsinin digərinin oxu və ya yazısından təsirlənməsidir.
Texniki izah: Read-write dependency serial order qərarına təsir edir.
Nümunə: A B-nin sonra dəyişdirdiyi məlumatı oxuyub.

Predicate lock bloklamır

Adına lock deyilsə də məqsədi növbə yaratmaq deyil. Təhlükəli dependency aşkar ediləndə PostgreSQL transaction-u abort edir.

26. Blocking, waiting və conflict

Üç ayrı anlayışın fərqi.

Blocking - bloklanma

Sadə izah: Bir transaction digərinin lock-u açmasını gözləyir.
Texniki izah: Statement wait vəziyyətində qalır.
Nümunə: B, A-nın FOR UPDATE etdiyi row-u dəyişmək istəyir.

Conflict - konflikt

Sadə izah: İki əməliyyatın eyni düzgünlük qaydasına zidd şəkildə toqquşmasıdır.
Texniki izah: Konflikt blocking, constraint error və ya serialization failure ilə həll oluna bilər.
Nümunə: İki request eyni masanı rezervasiya edir.

Abort

Sadə izah: Transaction-un commit etmədən dayandırılmasıdır.
Texniki izah: Database və ya tətbiq rollback tələb edir.
Nümunə: Serializable 40001 verdikdə transaction abort olur.

Mexanizm	Gözləyir?	Nəticə
FOR UPDATE	Çox vaxt bəli	Lock açıldıqdan sonra davam edir
Advisory blocking lock	Bəli	Açar boşalanda davam edir
Try advisory lock	Xeyr	Dərhal true/false
Serializable SSI	Adətən predicate üçün yox	Təhlükəli transaction 40001 ilə abort ola bilər

27. Row lock və FOR UPDATE

Mövcud konkret row-u dəyişiklik üçün kilidləmək.

Row lock

Sadə izah: Bir row üzərində zidd dəyişikliklərin növbələnməsidir.
Texniki izah: PostgreSQL row-level lock transaction sonuna qədər saxlanır.
Nümunə: Reservation #22 statusunu dəyişən transaction row-u kilidləyir.

SELECT FOR UPDATE

Sadə izah: Row-u oxuyub eyni zamanda onu dəyişiklik üçün kilidləyən query-dir.
Texniki izah: Digər UPDATE/DELETE/FOR UPDATE əməliyyatları gözləyə bilər.
Nümunə: SELECT * FROM reservations WHERE id=22 FOR UPDATE.

```
BEGIN;
SELECT * FROM reservations WHERE id = 22 FOR UPDATE;
-- status qaydasını yoxla
UPDATE reservations SET status_id = @confirmed WHERE id = 22;
COMMIT;
```

Nəyi qorumur?

FOR UPDATE yalnız tapılan mövcud row-u kilidləyir. Hələ mövcud olmayan gələcək rezervasiya slotunu təkbaşına kilidləyə bilməz.

28. Advisory lock

Tətbiqin mənasını verdiyi rəqəmsal resurs açarını kilidləmək.

Advisory lock

Sadə izah: Database-də row olmasa belə məntiqi resursu növbələndirən lock-dur.
Texniki izah: PostgreSQL açarı kilidləyir; açarın nə demək olduğunu tətbiq bilir.
Nümunə: (restaurantId, tableId) həmin masa üçün rezervasiya əməliyyatını serial edir.

Row lock	Advisory lock
Mövcud row lazımdır	Virtual resurs ola bilər
Database həmin row conflict-ini anlayır	Bütün kod yolları lock konvensiyasına əməl etməlidir
FOR UPDATE ilə alınır	pg_advisory_xact_lock ilə alına bilər
Status/balans row-u üçün uyğundur	Table/time slot və customer/day üçün uyğundur

Isolation ilə münasibət

Advisory lock isolation level deyil. ECafe-də Read Committed yeni snapshot verir, advisory lock isə eyni biznes resursu üçün request-ləri sıraya qoyur.

29. Optimistic və pessimistic concurrency

Konflikti əvvəl bloklamaq və ya sonra aşkarlamaq strategiyaları.

Pessimistic concurrency

Sadə izah: Konflikt olacaq deyə əvvəlcədən lock almaqdır.
Texniki izah: FOR UPDATE və advisory lock bu yanaşmaya daxildir.
Nümunə: Masa rezervasiyası yoxlanmadan əvvəl table lock alınır.

Optimistic concurrency

Sadə izah: Konflikt az olacaq deyə lock almadan işləyib yazı vaxtı dəyişiklik olub-olmadığını yoxlamaqdır.
Texniki izah: Version/concurrency token WHERE şərtinə əlavə edilir.
Nümunə: UPDATE ... WHERE id=22 AND version=5; 0 row dəyişdisə konflikt var.

Concurrency token

Sadə izah: Row-un oxunduqdan sonra dəyişib-dəyişmədiyini göstərən versiya dəyəridir.
Texniki izah: EF Core token-u UPDATE WHERE hissəsinə əlavə edir.
Nümunə: PostgreSQL xmin uyğun ssenaridə token kimi map edilə bilər.

Seçim

Konflikt tez-tezdirsə pessimistic lock daha proqnozlaşdırılan ola bilər. Konflikt nadirdirsə optimistic token daha yüksək concurrency verə bilər.

30. Atomic UPDATE və compare-and-set

Bəzi yarışları isolation artırmadan bir SQL ilə həll etmək.

Atomic operation

Sadə izah: Arada başqa əməliyyat girə bilməyəcək vahid database əməliyyatıdır.
Texniki izah: Tək UPDATE statement server tərəfində bir iş kimi icra olunur.
Nümunə: SET balance = balance - 30.

Compare-and-set

Sadə izah: Dəyəri yalnız gözlənilən əvvəlki vəziyyətdədirsə dəyişməkdir.
Texniki izah: UPDATE WHERE status_id = expectedStatus və affected row yoxlamasıdır.
Nümunə: Yalnız PaymentSubmitted statusundadırsa Confirmed et.

```
UPDATE reservations
SET status_id = @confirmed
WHERE id = @id
  AND status_id = @paymentSubmitted;

-- affected rows = 0 → vəziyyət artıq dəyişib
```

Affected rows

Sadə izah: SQL əməliyyatının neçə row dəyişdirdiyini göstərən saydır.
Texniki izah: 0 nəticəsi optimistic conflict və ya şərtin ödənməməsi ola bilər.
Nümunə: 0 row olduqda API 409 Conflict qaytara bilər.

31. Retry nədir?

Müvəqqəti concurrency xətasından sonra bütün transaction-u yenidən işlətmək.

Retry

Sadə izah: Uğursuz əməliyyatı yenidən sınamaqdır.
Texniki izah: 40001 və bəzi 40P01 hallarında bütün transaction funksiyası yenidən icra edilir.
Nümunə: Serializable conflict alan rezervasiya əməliyyatı yeni snapshot-la təkrar başlayır.

SQLSTATE

Sadə izah: PostgreSQL xəta növünün standart kodudur.
Texniki izah: Mesaj mətnindən daha sabit proqramlaşdırma interfeysidir.
Nümunə: 40001 serialization_failure, 40P01 deadlock_detected.

Kod	Məna	Adətən nə edilir?
40001	Serialization failure	Tam transaction retry
40P01	Deadlock detected	Siyasətə uyğun retry
23505	Unique violation	Biznes konflikti və ya diqqətli retry

Yalnız son query retry edilmir

Yeni snapshot və yeni biznes qərarı lazım olduğuna görə BEGIN-dən COMMIT-ə qədər bütün məntiq təkrar işləməlidir.

32. Idempotency və outbox

Retry zamanı xarici side effect-lərin təkrarlanmamasını təmin edən anlayışlar.

Side effect - yan təsir

Sadə izah: Database nəticəsindən kənarda dünyanı dəyişən əməliyyatdır.
Texniki izah: E-mail, push notification, bank çağırışı və fayl upload-u side effect-dir.
Nümunə: Müştəriyə “rezervasiya təsdiqləndi” notification-u göndərmək.

Idempotency

Sadə izah: Eyni əməliyyat təkrar icra edilsə də nəticənin əlavə dəfə yaranmamasıdır.
Texniki izah: Idempotency key və unique constraint dublikat side effect-i saxlayır.
Nümunə: Eyni payment request iki dəfə gəlsə yalnız bir ödəniş yaranır.

Outbox pattern

Sadə izah: Biznes dəyişikliyi ilə göndəriləcək mesajı eyni database transaction-da qeyd edib sonradan worker ilə göndərməkdir.
Texniki izah: Commit və message publication arasında etibarlılıq boşluğunu azaldır.
Nümunə: Reservation və NotificationOutbox row-u birlikdə commit olunur.

Retry-safe transaction

Serializable retry zamanı notification birbaşa transaction içində göndərilsə iki dəfə gedə bilər. Outbox + idempotent consumer bunu idarə edir.

33. Deadlock

Transaction-ların bir-birinin saxladığı lock-u qarşılıqlı gözləməsi.

Deadlock

Sadə izah: A B-ni, B də A-nı gözlədiyi üçün heç birinin davam edə bilmədiyi vəziyyətdir.
Texniki izah: PostgreSQL cycle-ı aşkarlayıb transaction-lardan birini 40P01 ilə abort edir.
Nümunə: A əvvəl customer sonra table; B əvvəl table sonra customer lock alır.

```
Transaction A: lock(Customer) → lock(Table) gözləyir
Transaction B: lock(Table)    → lock(Customer) gözləyir
```

Lock order

Sadə izah: Bir neçə lock-un bütün kod yollarında eyni sıra ilə alınması qaydasıdır.
Texniki izah: Cycle ehtimalını azaldır.
Nümunə: Həmişə əvvəl customer/day, sonra table/time lock.

Deadlock və serialization failure eyni deyil

Deadlock qarşılıqlı gözləmədir. Serialization failure isə serial nəticə mümkün olmadığı üçün abort qərarıdır. Hər ikisi retry tələb edə bilər, amma səbəbləri fərqlidir.

34. Savepoint, timeout və cancellation

Transaction-un idarəsi üçün köməkçi anlayışlar.

Savepoint

Sadə izah: Transaction daxilində qismən geri qayıtma nöqtəsidir.
Texniki izah: ROLLBACK TO SAVEPOINT bütün transaction-u bitirmir.
Nümunə: İkinci SaveChanges uğursuz olsa əvvəlki mərhələyə qayıtmaq.

Timeout

Sadə izah: Əməliyyata verilən maksimum gözləmə müddətidir.
Texniki izah: lock_timeout yalnız lock wait, statement_timeout statement müddəti üçün istifadə oluna bilər.
Nümunə: Lock 2 saniyədə alınmırsa kontrollu xəta.

Cancellation

Sadə izah: İstifadəçi və ya server request-in davam etməsini dayandırır.
Texniki izah: CancellationToken database əməliyyatına ötürülür və transaction rollback edilir.
Nümunə: Client bağlantını kəsəndə uzun query dayandırılır.

Savepoint yeni transaction deyil

Isolation snapshot və transaction-level lock-lar savepoint rollback-dan sonra da əsas transaction bitənə qədər davam edə bilər.

35. EF Core-da isolation seçimi

System.Data.IsolationLevel transaction başlayarkən verilir.

```
await using var tx = await db.Database.BeginTransactionAsync(
    IsolationLevel.ReadCommitted,
    cancellationToken);

try
{
    // lock, re-check, write
    await db.SaveChangesAsync(cancellationToken);
    await tx.CommitAsync(cancellationToken);
}
catch
{
    await tx.RollbackAsync(cancellationToken);
    throw;
}
```

DbContext

Sadə izah: EF Core-un entity-ləri izləyib database query və yazılarını idarə edən obyektidir.
Texniki izah: Connection və transaction ilə işləyir; thread-safe deyil.
Nümunə: ECafeDbContext reservation entity-lərini saxlayır.

SaveChanges

Sadə izah: DbContext-də yığılan dəyişiklikləri database-ə göndərir.
Texniki izah: Explicit transaction yoxdursa öz qısa transaction-ını yarada bilər.
Nümunə: Reservation INSERT əmri database-ə göndərilir.

Vacib

İki SaveChanges və aradakı query birlikdə atomik olmalıdırsa explicit transaction sərhədi yaradılmalıdır.

36. ECafe flow-u niyə işləyir?

Read Committed və advisory lock-un birlikdə rolu.

Addım	Nə olur?	Hansı anlayış işləyir?
1	Read Committed transaction başlayır	Hər query üçün təzə snapshot
2	Customer/day advisory lock alınır	Eyni invariant üçün request-lər növbələnir
3	Gündəlik rezervasiya yenidən yoxlanır	Lock-dan sonrakı re-check son commit-i görür
4	Table lock alınır	Eyni masa əməliyyatı serial olur
5	Availability yenidən yoxlanır	Köhnə check istifadə edilmir
6	Reservation yazılır və commit edilir	Atomic transaction; xact lock açılır

Read Committed-in faydası

B lock gözlədikdən sonra query göndərəndə yeni statement snapshot-u A-nın commit etdiyi rezervasiyanı görür.

Advisory lock-un faydası

A və B-nin check və write addımları eyni resurs üçün bir-birinə qarışmır.

Constraint-in faydası

Kod yolu səhvən lock-u bypass etsə, mümkün olan invariantlarda database son müdafiə xətti verir.

37. Hansı level-i seçməliyəm?

Əzbər cavab yox, qərar ağacı.

Sual	Bəli	Xeyr
Tək statement atomic həll mümkündür?	Read Committed + atomic SQL	Növbəti suala keç
Mövcud konkret row kilidlənə bilər?	Read Committed + FOR UPDATE	Növbəti suala keç
Aydın virtual biznes açarı varmı?	Read Committed + advisory lock + re-check	Növbəti suala keç
Sabit read-only görünüş lazımdır?	Repeatable Read	Növbəti suala keç
Cross-row invariant/write skew riski varmı?	Serializable + full retry	Read Committed default ola bilər

Best practice

Ən aşağı işlək isolation level-i seç, sonra invariantı ən konkret primitive ilə qoru: atomic UPDATE, constraint, row lock və ya advisory lock. Serializable seçirsənsə retry və idempotency-ni əvvəlcədən qur.

38. Tam müqayisə cədvəli

PostgreSQL-in real davranışı.

Xüsusiyyət	Read Uncommitted	Read Committed	Repeatable Read	Serializable
PG-də real davranış	Read Committed	Ayrı level	Snapshot Isolation	SSI
Snapshot	Hər statement	Hər statement	Transaction boyu	Transaction boyu
Dirty read	Yox	Yox	Yox	Yox
Non-repeatable	Mümkün	Mümkün	Yox	Yox
Phantom	Mümkün	Mümkün	Yox	Yox
Write skew	Mümkün	Mümkün	Mümkün	Commit edilən nəticədə yox
Retry	Adətən level səbəbilə yox	Adətən level səbəbilə yox	40001 mümkündür	40001 gözləniləndir
Tipik istifadə	Seçməyə ehtiyac yoxdur	Qısa API flow	Sabit hesabat	Mürəkkəb invariant

Cədvəlin açarı

"Yox" database-in həmin anomaliyanı commit olunmuş nəticədə buraxmadığını göstərir. "Mümkün" isə hər dəfə baş verəcək demək deyil; uyğun parallel ssenaridə baş verə bilər.

39. Praktik test planı

Isolation davranışı iki ayrı connection ilə yoxlanmalıdır.

Integration test

Sadə izah: Real database və bir neçə komponentlə davranışı yoxlayan testdir.
Texniki izah: Isolation və lock testləri in-memory provider ilə etibarlı ölçülmür.
Nümunə: İki Npgsql connection eyni PostgreSQL test database-də işləyir.

Test	A nə edir?	B nə edir?	Gözlənilən
Non-repeatable	SELECT → gözlə → SELECT	UPDATE + COMMIT	RC dəyişir, RR sabitdir
Phantom	COUNT → gözlə → COUNT	INSERT + COMMIT	RC artır, RR sabitdir
Write skew	Invariant oxu + row A write	Invariant oxu + row B write	RR keçirə bilər, Serializable biri 40001
Advisory lock	Lock + write	Eyni lock + check	B gözləyir və yeni vəziyyəti görür

Deterministic test

Task.Delay-ə güvənmək əvəzinə Barrier və ya TaskCompletionSource ilə addımların sırasını idarə et. Son HTTP cavabla yanaşı database vəziyyətini də assert et.

40. Terminlər lüğəti A-I

Sənəddə işlənən sözlərin qısa, müstəqil izahı.

Termin	Tam sadə izah
Abort	Transaction-un commit etmədən dayandırılması və rollback tələb etməsi
ACID	Atomicity, Consistency, Isolation, Durability prinsiplərinin ümumi adı
Affected rows	SQL yazı əməliyyatının neçə row dəyişdirdiyi
Anomaly	Concurrency səbəbindən yaranan arzuolunmaz nəticə
Atomic	Araya başqa əməliyyat daxil olmadan vahid şəkildə icra olunan
Blocking	Bir əməliyyatın digərinin lock-u açmasını gözləməsi
Commit	Transaction dəyişikliklərinin təsdiqlənməsi
Concurrency	Bir neçə əməliyyatın zaman baxımından üst-üstə düşməsi
Conflict	İki əməliyyatın eyni resurs və ya qayda üzərində toqquşması
Constraint	Database-in məcburi məlumat qaydası; məsələn UNIQUE və CHECK
Deadlock	Transaction-ların bir-birini qarşılıqlı gözləməsi
Dirty read	Başqasının commit olunmamış məlumatını oxumaq
Idempotency	Təkrar icranın əlavə nəticə yaratmaması
Invariant	Sistemdə həmişə doğru qalmalı biznes qaydası
Isolation level	Transaction-ların bir-birinin dəyişikliklərini görmə və konflikt qaydası

41. Terminlər lüğəti L-R

Lock-dan retry-yə qədər.

Termin	Tam sadə izah
Latency	Əməliyyatın cavab verməsinə sərf olunan vaxt
Lock	Zidd əməliyyatları məhdudlaşdıran koordinasiya mexanizmi
Lock order	Çoxlu lock-ların həmişə eyni sıra ilə alınması qaydası
Lost update	Bir concurrent yazının digərinin nəticəsini itirməsi
MVCC	Eyni row-un versiyaları ilə paralel oxu-yazını idarə edən model
Non-repeatable read	Eyni row-u ikinci oxuda fərqli görmək
Optimistic concurrency	Konflikti yazı zamanı version token ilə aşkarlamaq
Outbox	Göndəriləcək mesajı biznes dəyişikliyi ilə eyni transaction-da qeyd etmək
Phantom read	Eyni filter query-də row siyahısının dəyişməsi
Predicate	Row-ları seçən filter şərti
Predicate lock	Serializable dependency analizi üçün oxunan şərtin qeyd edilməsi
Pessimistic concurrency	Konfliktdən əvvəl lock alıb əməliyyatları növbələmək
Race condition	Nəticənin əməliyyatların təsadüfi sırasından asılı olması
Rollback	Transaction dəyişikliklərinin geri atılması
Row lock	Konkret mövcud row üzrə zidd yazıları növbələyən lock

42. Terminlər lüğəti S-Z

Snapshot-dan write skew-a qədər.

Termin	Tam sadə izah
Savepoint	Transaction daxilində qismən geri dönüş nöqtəsi
Serialization anomaly	Yekun nəticənin heç bir ardıcıl transaction sırasına uyğun gəlməməsi
Serialization failure	Database-in təhlükəli transaction-u 40001 ilə dayandırması
Session	PostgreSQL bağlantısının server tərəfindəki vəziyyəti
Side effect	Database-dən kənarda e-mail, ödəniş və fayl kimi dəyişiklik
Snapshot	Transaction-un görə bildiyi row versiyalarının database görünüşü
SQLSTATE	PostgreSQL xəta növünün standart proqramlaşdırma kodu
SSI	Serializable Snapshot Isolation; təhlükəli read/write dependency-ləri aşkarlayan model
Statement	Database-ə göndərilən bir SQL əmri
Throughput	Sistemin müəyyən vaxtda tamamladığı əməliyyat sayı
Timeout	Əməliyyatın maksimum gözləmə müddəti
Transaction	Birlikdə commit və ya rollback olan database iş vahidi
Tuple version	Row-un müəyyən zamandakı konkret versiyası
Visibility	Snapshot-un hansı row versiyasını göstərməsi
Write skew	Fərqli row yazılarının ümumi invariantı birlikdə pozması

43. Tez-tez verilən suallar

Mövzunun ən çox qarışan hissələri.

Sual	Qısa cavab
Repeatable Read-də hər şey təhlükəsizdir?	Xeyr. Snapshot sabitdir, amma write skew mümkündür.
Serializable bütün query-ləri kilidləyir?	Xeyr. PostgreSQL SSI ilə dependency izləyir və lazım olsa abort edir.
Read Committed pis level-dir?	Xeyr. Qısa API flow-larında konkret lock/constraint ilə doğru default-dur.
Isolation artırmaq double booking-i avtomatik həll edir?	Həmişə yox. Constraint, advisory lock və retry dizaynı yenə lazımdır.
40001 server bug-ıdır?	Xeyr. Serializable/Repeatable Read protokolunun gözlənilən nəticəsidir.
Retry yalnız SaveChanges üçündür?	Xeyr. Qərar verən bütün transaction təkrar edilməlidir.
Phantom yoxdursa serialization anomaly niyə olur?	Phantom görünüşlə bağlıdır; anomaly transaction-ların birgə yekun nəticəsi ilə.
FOR UPDATE ilə advisory lock eynidir?	Xeyr. Biri mövcud row-u, digəri tətbiq açarını kilidləyir.

Bir cümləlik yekun

Isolation level transaction-un nə gördüyünü və konfliktə necə reaksiya verdiyini müəyyən edir; biznes düzgünlüyü isə level, lock, constraint, retry və idempotency birlikdə qurulduqda yaranır.

44. Rəsmi mənbələr

Bələdçinin texniki davranışı bu rəsmi sənədlərə əsaslanır.

PostgreSQL - Transaction Isolation
https://www.postgresql.org/docs/current/transaction-iso.html

Read Committed, Repeatable Read, Serializable, anomaliya cədvəli və SSI davranışı.

PostgreSQL - Serialization Failure Handling
https://www.postgresql.org/docs/current/mvcc-serialization-failure-handling.html

40001, 40P01 və bütün transaction-un retry qaydası.

PostgreSQL - Explicit Locking
https://www.postgresql.org/docs/current/explicit-locking.html

Row, table, deadlock və advisory lock davranışı.

Microsoft - Transactions in EF Core
https://learn.microsoft.com/ef/core/saving/transactions

EF Core transaction, savepoint və cross-context qaydaları.

Npgsql - Basic Usage / Transactions
https://www.npgsql.org/doc/basic-usage.html#transactions

Npgsql transaction və isolation API-si.

Son nəticə

Terminləri əzbərləmək lazım deyil. Hər dəfə iki transaction-un nə oxuduğunu, nə yazdığını və hansı invariantın qorunmalı olduğunu kağız üzərində addım-addım çəkmək lazımdır.